🚀 Booking Service - Complete Setup Guide

This guide will walk you through setting up your Booking Service application from scratch.

✅ What's Already Configured

- ✅ `.env.local` created with your Supabase credentials
- ✅ Git repository initialized and connected to `https://github.com/Nelsobral/BookingSlots`
- ✅ Initial commit ready to push
- ✅ All code files generated and ready

📋 Setup Steps

Step 1: Push to GitHub

The code is committed locally. Push it to your GitHub repo:

```
cd /home/ubuntu/booking-service
git push -u origin main
```

Step 2: Set Up Supabase Database

Go to your Supabase project dashboard: **https://ehpeqaqfxolmctkdwqwe.supabase.co**

Option A: Run Complete Setup (Recommended)

1. Go to **SQL Editor** in the left sidebar
2. Click **New Query**
3. Open the file `/home/ubuntu/booking-service/supabase/setup-complete.sql`
4. Copy the entire contents and paste into the SQL Editor
5. Click **Run** (or press Cmd/Ctrl + Enter)

This will create:
- All database tables (13 tables)
- All indexes and triggers
- Row Level Security (RLS) policies
- Helper functions for tenant isolation
- Demo data for "Luxe Beauty Studio"
- A demo login account

Option B: Run Separately (Advanced)

If you prefer to run schema and seed data separately:

1. **Schema**: Run `supabase/migrations/001_initial_schema.sql`
2. **Seed Data**: Run `supabase/seed.sql`

Step 3: Test Demo Login

After running the SQL setup, you can log in with the demo account:

- **Email**: `owner@luxebeauty.demo`
- **Password**: `DemoPassword123!`

This account owns the “Luxe Beauty Studio” business with:
- 2 staff members (Sophie Laurent, Marcus Chen)
- 4 services (Haircut, Massage, Facial, Manicure)
- 3 clients
- 5 bookings

Step 4: Run Locally

```
cd /home/ubuntu/booking-service
npm install
npm run dev
```

Visit: **http://localhost:3000**

You should see the landing page. Click “Log in” and use the demo credentials above.

Step 5: Deploy to Vercel (Optional)

1. Go to vercel.com (https://vercel.com)
2. Click **New Project**
3. Import your GitHub repository: `Nelsobral/BookingSlots`
4. Add environment variables:
   `NEXT_PUBLIC_SUPABASE_URL=https://ehpeqaqfxolmctkdwqwe.supabase.co`
   `NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_1Gc-4QqkzFOKgI1_0lVliw_Eklw_61f`
   `NEXT_PUBLIC_APP_URL=https://your-vercel-domain.vercel.app`
   `DEFAULT_FROM_EMAIL=Nelsobral@gmail.com`
   `RESEND_API_KEY=your_resend_api_key_here`
5. Click **Deploy**

🔐 Security Notes

Row Level Security (RLS)

Every table has RLS policies that enforce tenant isolation:
- Businesses can only see their own data
- Clients can only see their own bookings
- All queries use `business_id` filtering at the database level

Demo Account Security

The demo account (`owner@luxebeauty.demo`) is for testing only. For production:
1. Sign up with your real email via the app’s signup page
2. Complete the onboarding wizard
3. Delete or disable the demo account in Supabase Auth

📊 Database Schema Overview

Core Tables

- **profiles**: User profiles (linked to auth.users)
- **businesses**: Tenant data (salons, spas, etc.)
- **business_members**: Users belonging to businesses
- **staff_members**: Service providers
- **services**: Offered services (haircut, massage, etc.)
- **clients**: Business's clients
- **bookings**: Appointments
- **availability_rules**: Weekly schedules
- **availability_exceptions**: Blocked dates/special hours
- **reminder_events**: Reminder tracking
- **notification_preferences**: Per-business notification config

Key Helper Functions

- `get_user_business_id()` : Returns the authenticated user's business_id
- `is_business_owner()` : Checks if user owns a business
- `is_business_member()` : Checks if user is a member of a business

🎨 Customizing the Sender Email

The default sender email is `Nelsobral@gmail.com`, but each business can customize this:

1. Log in to the dashboard
2. Go to **Settings** → **Notifications** tab
3. Update the "From Email" field
4. Click **Save Changes**

This email will be used for all reminders sent by that business (Phase 2 feature).

🗺️ What's Next: Phase 2

Once Phase 1 is running, we'll build:

Phase 2 Features

- **Public Booking Flow** (`/book/[businessSlug]`)
- Real-time slot availability engine
- Client self-service booking
- Server-side slot validation
- **Client Portal** (`/client/bookings` , `/client/profile`)
- View upcoming/past bookings
- Confirm or cancel appointments

- Profile management
- **Email Reminder System**
- Automatic reminders 24h before appointments (configurable)
- Confirm/Cancel actions in email
- Resend integration
- SMS support architecture (for future Twilio integration)
- **Calendar View** (`/app/calendar`)
- Week/month views
- Drag-and-drop rescheduling
- Visual availability overview
- **Advanced Analytics**
- No-show tracking and flagging
- Revenue forecasting
- Client retention metrics

🐛 Troubleshooting

“Error: Missing environment variables”

Make sure `.env.local` exists and contains all required variables. Run:

```
cat /home/ubuntu/booking-service/.env.local
```

“Database connection error”

Verify your Supabase URL and anon key in `.env.local` match your project.

“RLS policy error” or “No rows returned”

This usually means:
1. You haven’t run the database setup SQL yet, OR
2. The logged-in user doesn’t have a `business_member` record

Solution: Run the setup SQL (Step 2 above) or create a business via the onboarding wizard.

TypeScript errors

Run:

```
npm install
npx tsc --noEmit
```

All types should pass without errors.

📝 File Structure Reference

```
booking-service/
├── app/                          # Next.js App Router pages
│   ├── (auth)/                   # Auth pages (login, signup)
│   ├── (public)/                 # Public landing page
│   ├── app/                      # Protected dashboard routes
│   │   ├── dashboard/
│   │   ├── bookings/
│   │   ├── services/
│   │   ├── staff/
│   │   └── settings/
│   ├── onboarding/               # Business setup wizard
│   └── auth/callback/            # OAuth callback
├── components/
│   ├── ui/                       # Reusable UI primitives
│   ├── dashboard/                # Dashboard-specific components
│   ├── forms/                    # Form components
│   └── public/                   # Landing page sections
├── lib/
│   ├── supabase/                 # Supabase clients (browser/server/middleware)
│   ├── actions/                  # Server Actions
│   ├── validations/              # Zod schemas
│   ├── utils.ts                  # Utilities
│   └── constants.ts              # Constants
├── supabase/
│   ├── migrations/               # SQL migrations
│   ├── seed.sql                  # Demo data
│   └── setup-complete.sql        # All-in-one setup
├── types/
│   ├── database.ts               # Generated Supabase types
│   └── index.ts                  # Domain types
├── .env.local                    # Your local environment variables (gitignored)
├── .env.example                  # Template for environment variables
└── README.md                     # Project documentation
```

✅ Ready to Continue?

Once you've completed steps 1-4 above and can access the dashboard at localhost:3000, you're ready for **Phase 2**!

Let me know when you're ready and I'll start building the booking flow, client portal, and reminder system.